Session 09 — Team Work Task

Goal

Today's live coding fetched `tasks` from a real server. `users` is still the old hardcoded 3-person array from Session 08 — which is why most task owner names currently show "Unknown person" (the real tasks reference person ids 1–10, but the hardcoded list only has ids 1–3). Your job: fetch `users` from a real server too, using the exact same pattern you just watched built for `tasks`.

Starting state

You're starting from today's live-code checkpoint: `tasks` is fetched, with working loading/error/Retry. `users` is still `const users: User[] = [...]`, hardcoded, right in `App.tsx`.

What to build

1. In `App.tsx`, add whatever this file needs to fetch the real people list — look at how it already fetches tasks for the shape to follow. The real people API is: `https://jsonplaceholder.typicode.com/users`
2. In `App.tsx`, replace the hardcoded `users` array with real fetched state, tracking loading and error the same way `tasks` does.
3. Trigger the users fetch at the same moment the tasks fetch already starts.
4. Show a "Loading people..." message while users are loading, and a clear error message with a working Retry button if the users fetch fails — reuse the CSS classes already built for tasks; no new CSS is needed.
5. Make sure `AddTaskForm`'s owner dropdown and every rendered owner name actually use the real fetched people once they arrive.
6. **Think about what happens the moment the page first loads, before `users` has arrived — it starts as an empty list.** `AddTaskForm` currently assumes there's always at least one real user to default to. Figure out what needs to change so the form doesn't break during that brief window, without hiding or removing the form.

Rules

Allowed / already known: everything from today's live coding (`useState`, `useEffect` with `[]`, `async` / `await`, `try` / `catch`, callback props, conditional rendering, `.find()`), plus everything from Sessions 05–08.

Not yet / do not use: a second `useEffect` with a non-empty dependency array, hiding/removing `AddTaskForm` from the page while people are loading, editing a task's owner, delete confirmation, keyboard event handling, props destructuring, React Fragment, extracting types into a separate `types.ts` file.

`tasks` and `users` must fail **independently** — a broken people fetch must never affect whether tasks load or display, and vice versa.

Test it

To actually trigger an error, use a genuinely broken (non-resolving) address, like `https://this-does-not-exist.invalid/...` — an ordinary "wrong path" URL that the server still responds to (even with a 404) will NOT trigger your `catch` block, since `fetch()` only rejects on a real network failure.

- Reload the app — you should briefly see both "Loading tasks..." and "Loading people...", then real tasks and real people.
- Owner names should now show real people's names instead of "Unknown person" for any task whose `userId` matches a real fetched person.
- Break only the people URL — tasks should still load and work completely normally; people should show an error message with a working Retry.
- Break only the tasks URL — the reverse should be true.
- Add a new task immediately after the page loads (as fast as you can) — it should not crash the app, even if people haven't finished loading yet.
- Toggle, Delete, and Edit should all still work exactly as before.

Done when

- □ Real people are fetched, with independent loading, error, and Retry.
- □ `AddTaskForm`'s dropdown and every owner name use real fetched people.
- □ Breaking the tasks fetch never affects people, and breaking the people fetch never affects tasks.
- □ The app never crashes, even if a task is added before people finish loading.
- □ Toggle, Delete, and Edit continue to work correctly.
- □ `npm run build` passes with no errors.

Hint

`AddTaskForm` currently picks a default owner with something like "use the selected person, or otherwise the first person in the list." What's the first thing that could go wrong with "the first

person in the list" the moment that list is empty? You already know how to guard against reading a property of something that might not exist — you've used array-length checks and `if` statements for exactly this kind of situation before.